

Project Report

Cache-Aware and Cache-Oblivious Algorithms for String Sorting

Kausheya Roy Harshit Lalwani

May 2026

1 Problem Statement

This project studies cache-aware and cache-oblivious algorithms with the goal of designing and implementing efficient solutions for **String Sorting**. We evaluate these algorithms based on their ability to minimize I/O operations $Q(N; M; B)$ between a fast internal memory of size M and a larger memory, where data is transferred in blocks of size B . The primary metric of success is how closely an algorithm approaches the scanning bound $Scan(N) = O(N/B)$.

2 The Architecture Problem with Baseline Algorithms

Standard algorithms often ignore the memory hierarchy, leading to significant performance degradation on large datasets.

2.1 Standard Introsort (`std::sort`)

Standard implementations like `std::sort` treat strings as opaque objects. When sorting N strings, the algorithm performs $O(N \log N)$ comparisons. In a string context, each comparison can trigger multiple cache misses as the CPU fetches character data from arbitrary heap locations, leading to poor spatial locality and high $Q(N)$.

2.2 Multi-key Quicksort (MKQS)

Idea: MKQS partitions strings into three sets (less than, equal to, and greater than) based on the character at the current depth d .

Problem: While MKQS avoids re-comparing characters, it suffers from poor cache utilization during partitioning. If the dataset size $N > M$, the random-access nature of the swaps causes frequent cache line evictions.

Algorithm 1 Multi-key Quicksort (A, lo, hi, d)

```
1: if  $hi \leq lo$  then return
2: end if
3:  $pivot \leftarrow char\_at(A[lo], d)$ 
4: Partition  $A[lo \dots hi]$  into  $\{< pivot, = pivot, > pivot\}$ 
5: MKQS $(A, lo, lt - 1, d)$ 
6: MKQS $(A, lt, gt, d + 1)$   $\triangleright$  Move to next character for equal partition
7: MKQS $(A, gt + 1, hi, d)$ 
```

3 Cache-Aware Algorithm: Burstsor

Idea: Burstsor uses a Trie where leaves are containers (buckets). It ensures the "Working Set" of strings being sorted fits into the cache.

Algorithm 2 Burstsor (S, L)

```
1: Initialize Trie root with an empty bucket
2: for each string  $s \in S$  do
3:   Traverse Trie to find the correct leaf bucket
4:   Insert  $s$  into bucket
5:   if  $size(bucket) > L$  then Burst(bucket)
6:   end if
7: end for
8: Sort each bucket using a cache-efficient baseline (e.g., MKQS)
```

Analysis:

- **TC:** $O(D + N \log L)$, where D is the total length of distinguishing prefixes.
- **Q Calculation:** $Q(N) \approx O(N/B \cdot D/L)$.
- **Proof:** By setting $L < M$, each bucket sort occurs entirely within the cache. The I/O cost is dominated by the initial distribution (effectively a Scan) and the final sequential read of the Trie, both of which are $O(N/B)$.

4 Cache-Aware Algorithm: MSD Radix Sort

Idea: MSD Radix Sort uses a distribution-based approach, counting frequencies of characters at depth d and shuffling strings into R buckets.

Analysis:

- **Q Calculation:** $Q(N) = O(N \cdot D/B)$ if $Alphabet_Size < M/B$.
- **Proof:** Each pass is a $Scan(N)$, costing $O(N/B)$. However, if the alphabet size is too large, the write pointers for each bucket will reside in different cache lines. If the number of lines exceeds M/B , every write may cause a cache miss (thrashing).

Algorithm 3 MSD Radix Sort (A, lo, hi, d, aux)

- 1: **Count:** Frequencies of $char_at(s, d)$ for $s \in A[lo \dots hi]$
 - 2: **Indices:** Transform counts into starting positions in aux
 - 3: **Distribute:** Move strings from A to aux using indices
 - 4: **Copy Back:** $A[lo \dots hi] \leftarrow aux[0 \dots hi - lo]$
 - 5: **Recurse:** For each character $r \in \text{Alphabet}$, sort sub-partitions at $d + 1$
-

5 Cache-Oblivious Algorithm: Funnelsort (Lazy)

Idea: Uses a recursive divide-and-conquer approach that is optimal without knowing M or B . We implement a multi-way merge sort that recursively splits the input.

Algorithm 4 Lazy Funnelsort (S)

- 1: **if** $\text{size}(S)$ fits in cache block B **then**
 - 2: Sort S using a standard base-case sort
 - 3: **else**
 - 4: Divide S into $K = N^{1/3}$ streams
 - 5: **Lazy Funnelsort**(each stream)
 - 6: Merge streams using a K -way Funnel (merger)
 - 7: **end if**
-

Analysis:

- **Q Calculation:** $Q(N) = O(N/B \log_{M/B} N/B)$.
- **Proof:** Because the algorithm uses divide-and-conquer, it eventually reaches a subproblem size n such that $n^2 \leq M$. At this point, the subproblem and its merge buffers fit entirely in the cache, and all subsequent recursive calls incur zero further I/O misses. This recursive structure makes it optimal for any M and B .

6 Datasets

To rigorously test the architecture-friendliness, we utilized two primary dataset categories:

6.1 Synthetic Datasets

Generated using C++ pseudo-random generators to control specific performance variables.

- **Random:** Strings of lengths 4–24 with uniform character distribution to test $N \log N$ behaviors.

- **Prefix-Heavy:** Strings sharing long common prefixes (e.g., "cache_aware_1", "cache_aware_2") to force algorithms to process deeper distinguishing prefixes (D) and stress the Q complexity.

6.2 Real-World Dataset: Wikipedia Titles

A collection of 5 million titles from English Wikipedia. This dataset represents real-world "Long Tail" distributions and heavy prefix overlaps, providing a benchmark for how algorithms handle actual data skewedness.

7 Experiments and Performance Analysis

We conducted benchmarks across varying problem sizes $N = \{10^5, 10^6, 5 \times 10^6\}$.

7.1 Time Performance per Element

We plotted the execution time per element (in μs) against N for all algorithms. Normalizing by N reveals cache-related performance degradation as problem sizes grow.

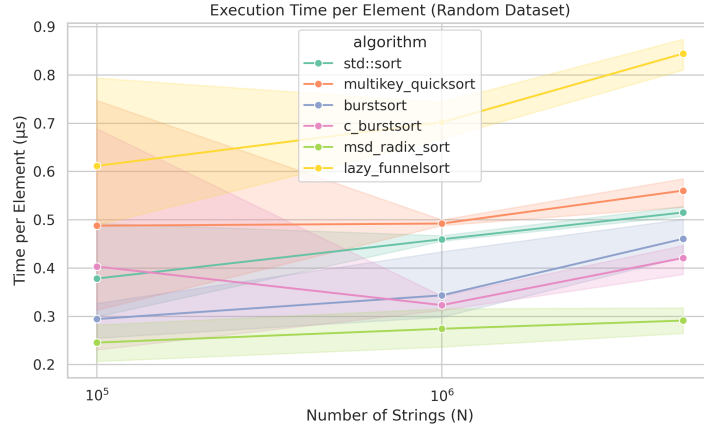


Figure 1: Execution Time per Element on Random Dataset.

Observation: Cache-aware algorithms like Burtsort and C-Burtsort maintain relatively flat scaling (constant time per element) for large N , whereas standard baseline sorts and memory-intensive approaches like MSD Radix Sort exhibit noticeable performance degradation as N exceeds the LLC capacity M .

7.2 Cache Profiling (L1 and LLC Misses)

We utilized hardware performance counters to measure L1 Data Misses and LLC (Last Level Cache) Data Misses per element.

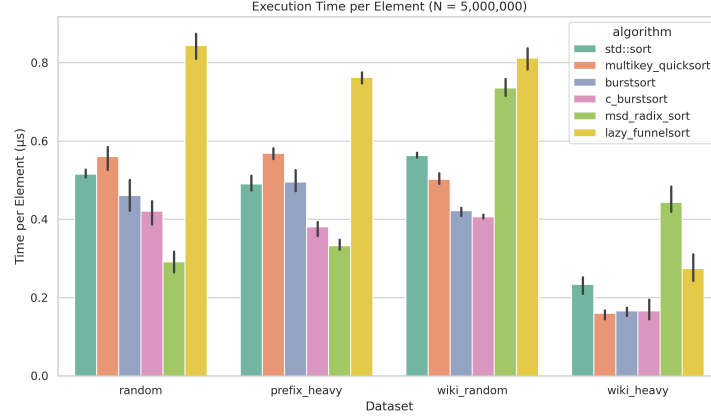


Figure 2: Execution Time per Element across all datasets for $N = 5 \times 10^6$.

Observation: MSD Radix Sort shows significantly higher miss rates due to bucket thrashing, while Burstersort remains near the theoretical scanning limit, demonstrating its superior cache utilization.

8 Codebase Architecture

The project is implemented in **C++20** for high-performance memory management and deterministic cache control.

8.1 File Structure

- **main.cpp:** Entry point; manages dataset generation, loading, and the benchmarking loop.
- **algorithms/:** Contains implementation headers and source files.
 - **radix.cpp:** MSD Radix Sort implementation.
 - **burstersort.cpp:** Optimized C-Burstersort using an arena-based memory model.
 - **baselines.cpp:** implementations of MKQS and `std::sort` wrappers.
- **Analysis/:** Python scripts (`update_nbs.py`) and Jupyter notebooks for data visualization using Seaborn and Matplotlib.

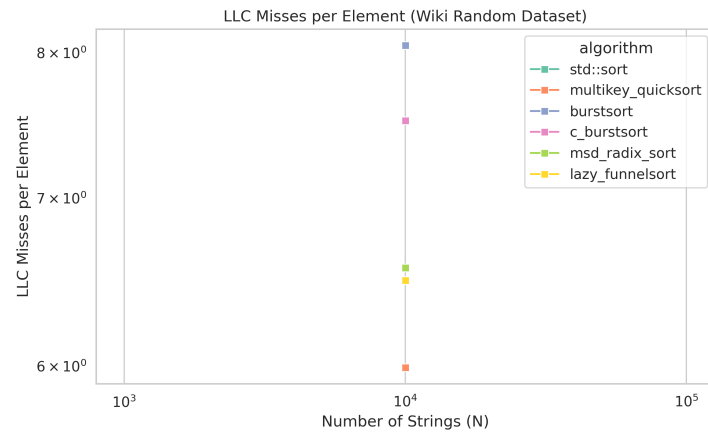


Figure 3: LLC Misses per Element on Wiki Random Dataset.